

Feuille de travaux dirigés n° 1

Du parallélisme à la concurrence

— CORRECTION —

Exercice 1 (La loi d'Amdahl).

Une proportion $p = 70\%$ d'un programme séquentiel peut être parallélisée. On dispose d'une machine avec $n = 10$ cœurs.

1a. Quelle est l'accélération (speedup) attendue ?

$$S(10) = \frac{1}{(1-p) + \frac{p}{10}} = \frac{1}{0,3 + \frac{0,7}{10}} \approx 2,70.$$

1b. On trouve dans le commerce une machine avec $n = 20$ cœurs. Quelle est l'accélération attendue ?

$$S(20) = \frac{1}{(1-p) + \frac{p}{20}} = \frac{1}{0,3 + \frac{0,7}{20}} \approx 2,99.$$

1c. Quelle est l'accélération maximale théorique lorsque $n \rightarrow \infty$?

$$\lim_{n \rightarrow \infty} S(n) = \frac{1}{1-p} = \frac{1}{0,3} \approx 3,33.$$

Un ingénieur pense qu'il existe un moyen d'optimiser la partie séquentielle pour la rendre deux fois plus rapide.

1d. Quelle proportion du programme est parallélisable dans la version optimisée ?

On suppose maintenant que la partie séquentielle est deux fois plus rapide, mais que la partie non parallélisable est inchangée. On obtient donc :

$$T_1^{\text{opt}} = \frac{T_1 \times (1-p)}{2} + T_1 \times p.$$

La proportion parallélisable est donc :

$$p_{\text{opt}} = \frac{T_1 \times p}{T_1^{\text{opt}}} = \frac{2p}{1+p} \approx 82\%.$$

1e. Quelle accélération obtient-on alors, dans la parallélisation avec $n = 10$ cœurs ?

$$S_{\text{opt}}(10) = \frac{1}{(1 - p_{\text{opt}}) + \frac{p_{\text{opt}}}{10}} \approx 3,86.$$

1f. À coût égal, vaut-il mieux choisir d'acheter la machine avec $n = 20$ cœurs ou payer l'ingénieur pour optimiser le code ?

Il faut comparer l'accélération par rapport à T_1 , de T_{20} , le temps d'exécution du programme initial sur la machine à 20 cœurs, et de T_{10}^{opt} , le temps d'exécution du programme optimisé sur la machine à 10 cœurs.

$$- \frac{T_{20}}{T_1} = S(20) \approx 2,99.$$

$$- \frac{T_{10}^{\text{opt}}}{T_1} = \frac{1-p}{2} + \frac{p}{10} \approx 4,55.$$

L'optimisation de la partie séquentielle est nettement plus rentable.

Exercice 2 (Parallélisation).

Le but de cet exercice est d'écrire une fonction `double[][] product(double[][] M1, double[][] M2)` qui calcule le produit de deux matrices carrées M_1 et M_2 de taille $n \times n$. On rappelle la définition de la multiplication des matrices. Si $M_3 = M_1 \times M_2$, alors pour tout $i, j \in \{0, \dots, n-1\}$,

$$M_3[i][j] = \sum_{k=0}^{n-1} M_1[i][k] \times M_2[k][j].$$

2a. Implémentez la version séquentielle de `product`.

```
public interface Product {
    double[][] product(double[][] M1, double[][] M2) throws InterruptedException;
}
public final class Sequential implements Product {
    double[][] product(double[][] M1, double[][] M2) {
        int n = M1.length;
        double[][] M3 = new double[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                M3[i][j] = 0;
                for (int k = 0; k < n; k++) {
                    M3[i][j] += M1[i][k] * M2[k][j];
                }
            }
        }
        return M3;
    }
}
```

2b. Proposez une parallélisation où chaque cellule $M_3[i][j]$ est calculée par un thread distinct (jusqu'à n^2 threads).

```
package tdl_parallelism.matrix;

public final class Naive implements Product {

    public double[][] product(double[][] M1, double[][] M2) throws InterruptedException {

        int n = M1.length;
```

```

double[][] M3 = new double[n][n];
Thread[] threads = new Thread[n*n];

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        final int ii = i, jj = j;
        threads[i*n+j] = new Thread(() -> {
            M3[ii][jj] = 0;
            for (int k = 0; k < n; k++) {
                M3[ii][jj] += M1[ii][k] * M2[k][jj];
            }
        });
    }
}

for(var t : threads) t.start();
for(var t : threads) t.join();

return M3;
}

```

2c. Proposez une version qui crée au plus T threads (T est une variable globale), chacun calculant un lot de cellules. Précisez votre stratégie de répartition.

```

public static double[][] product(int n, double[][] M1, double[][] M2) {
    double[][] M = new double[n][n];
    Thread threads[][] = new Thread[n][n];
    for(int i = 0; i < n; i++) {
        int thisI = i;
        threads[i][0] = new Thread(() -> {
            for(int j = 0; j < n; j++) {
                int thisJ = j;
                threads[thisI][j] = new Thread(() -> {
                    M[thisI][thisJ] = 0;
                    for(int k = 0; k < n; k++) {
                        M[thisI][thisJ] += M1[thisI][k] * M2[k][thisJ];
                    }
                });
            }
        });
        threads[thisI][j].start();
    }
    for(int j = 1; j < n; j++) {
        try {
            threads[thisI][j].join();
        } catch (InterruptedException e) { }
    }

    });
    threads[i][0].start();
}
for(int i = 1; i < n; i++) {
    try {
        threads[i][0].join();
    } catch (InterruptedException e) { }
}
return M;
}

```

2d. Analysez la complexité en temps des versions et justifiez l'intérêt de limiter le nombre de threads à une borne indépendante de n .

Si on parallélise la boucle intérieure, on passe d'une complexité $O(n^3)$ à $O(n^2)$, soit une accélération en $O(n)$. On peut faire mieux en parallélisant la boucle extérieure également. On passe alors de $O(n^3)$ à $O(n)$, soit une accélération en $O(n^2)$. Cela ne viole pas la loi d'Amdahl car on change le problème en changeant le nombre de threads : on augmente le nombre de threads pour s'adapter à la quantité de données à traiter. C'est l'idée de la loi de Gustafson : Si le nombre de threads est proportionnel à la taille des données, on peut espérer une accélération en $O(n)$.

Remarque : le programme ci-dessus utilise $n^2 + n + 1$ threads, mais on peut utiliser les mêmes threads pour paralléliser et pour faire certains calculs.

Exercice 3 (Asynchronie).

On se donne le programme multi-threads suivant :

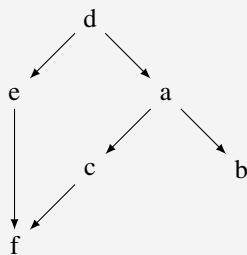
```
Thread t = new Thread() -> {  
    System.out.print("a");  
    new Thread() -> { System.out.print("b"); }.start();  
    System.out.print("c");  
};  
  
System.out.print("d");  
t.start();  
System.out.print("e");  
t.join();  
System.out.print("f");
```

3a. Décrivez la relation “happens-before” du programme.

3b. Décrivez toutes les sorties possibles pour le programme, en sachant qu’il est séquentiellement cohérent.

3c. Décrivez la machine à états du programme.

Les sorties possibles sont toutes les linéarisations du graphe de causalité, soit les 11 sorties suivantes :



- deacfb
- deacbf
- deabcf
- daecfb
- daecbf
- daebcf
- dabecf
- dacefb
- dacebf
- dacbef
- dabcef

Exercice 4 (Le savant fou ¹).

En visite pédagogique dans un laboratoire, vous et le reste de votre groupe êtes capturés par un savant fou. Il fait l’annonce suivante :

« J’ai mis en place une pièce qui contient une lumière et un interrupteur qui la commande. Actuellement, la lumière est éteinte. Vous pouvez discuter ce soir d’une stratégie commune, mais à partir de demain vous serez isolés et ne pourrez plus communiquer. Chaque jour, je tirerai au hasard de manière uniforme l’un d’entre vous, qui passera la journée dans la pièce. À tout moment durant sa visite, il pourra déclarer : *Nous avons tous visité la pièce*. S’il a raison, vous êtes libres. Sinon, je vous reboote les cerveaux ! »

Hypothèses implicites :

- La lumière est éteinte au départ.
- L’interrupteur est binaire (allumé/éteint).
- La lumière est invisible depuis l’extérieur de la pièce, et il n’y a aucun autre moyen de communication entre les étudiants.

4a. Énoncez précisément ce que doit garantir une stratégie réussie.

1. Inspiration : M. Herlihy & N. Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann (2008)

Une stratégie correcte doit vérifier :

Sûreté. Si un étudiant déclare « Nous avons tous visité la pièce », alors tous les étudiants l'ont effectivement visitée.

Vivacité. Avec probabilité 1, un étudiant finira par pouvoir faire cette déclaration.

4b. Proposez une stratégie et justifiez brièvement en quoi elle satisfait l'exigence précédente.

Une stratégie possible consiste à désigner un étudiant comme *délégué* :

- Chaque étudiant non délégué allume la lumière la première fois qu'il entre dans la pièce et la trouve éteinte, puis ne la rallume plus jamais.
- Chaque fois que le délégué entre dans la pièce et voit la lumière allumée, il l'éteint et incrémente un compteur local.
- Lorsque son compteur atteint $n - 1$, il déclare que tous les étudiants ont visité la pièce.

Cette stratégie garantit la sûreté (le délégué ne peut compter qu'un allumage par étudiant) et la vivacité presque sûrement (le tirage aléatoire finit par sélectionner chaque étudiant au moins une fois alors que la lumière est éteinte).

4c. Reprenez la question précédente, sans l'hypothèse sur l'état initial de la lumière.

Si l'état initial de la lumière est inconnu, le protocole peut être adapté en demandant au délégué d'ignorer le premier allumage observé (pour éliminer l'incertitude sur l'état de départ).

4d. Reprenez les deux premières questions, sans l'hypothèse que le tirage est uniforme. À la place, on suppose que si personne ne déclare que tout le monde a visité la pièce, chaque étudiant est sélectionné une infinité de fois (pas nécessairement à intervalles bornés).

Si le tirage n'est pas uniforme mais seulement équitable (chaque étudiant est choisi une infinité de fois, sans borne sur l'intervalle entre deux visites), la stratégie reste correcte : la sûreté n'est pas affectée, et la vivacité est renforcée et devient déterministe : un étudiant finira par pouvoir faire cette déclaration.